DFT exercises with VASP
a hands-on course by Dominik Juraschek

One solution proposed by Verena Brehm,
in collaboration with Daniel Bustamente and Yu-Chi Huang

July 2025

Introductory comments

With four exercises we will learn how to use the code VASP for doing simple DFT calculations. Instructions can be found on the exercise sheets created by Dominik Juraschek. The input files, which are called INCAR, KPOINTS, POSCAR and POTCAR are provided and specific for each exercise, and the former three need to be adapted according to the task. All material can be found on my github `https://github.com/vjbNO/DFTexercises`. The calculations have been run on the TU/e cluster Umbrella.

Hand-written notes of an introductory lecture held by Dominik are also uploaded there. A book recommended by Alireza : *Time-Dependent Density-Functional Theory: Concepts and Applications* by Carsten A. Ullrich (first chapter gives nice general intro to (not-time-dependent) DFT).

1 Band structure and DOS of Si

First we will do some basic things to learn how to use the code. You always need the input files INCAR, KPOINTS, POSCAR and POTCAR in the same directory as the jobscript, and they may not be called differently.

1.1 Convergence testing

To see which energy cut, how many points in reciprocal space (KPOINTS) and which lattice constant to use, we calculate the total free energy while scanning ENCUT (in the INCAR), n KPOINTS (in the KPOINTS file) and the lattice constant (first value in the POSCAR file). For Si, I find that an ENCUT$>$ 600, nKPOINTS$>$ 10 and a lattice constant of 5.4 Å seem appropriate.

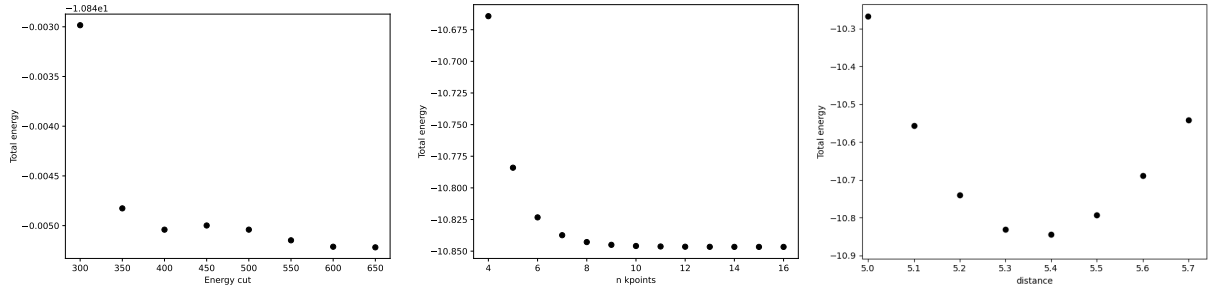

Figure 1: Energy convergence, n KPOINTS and geometry optimization for Si

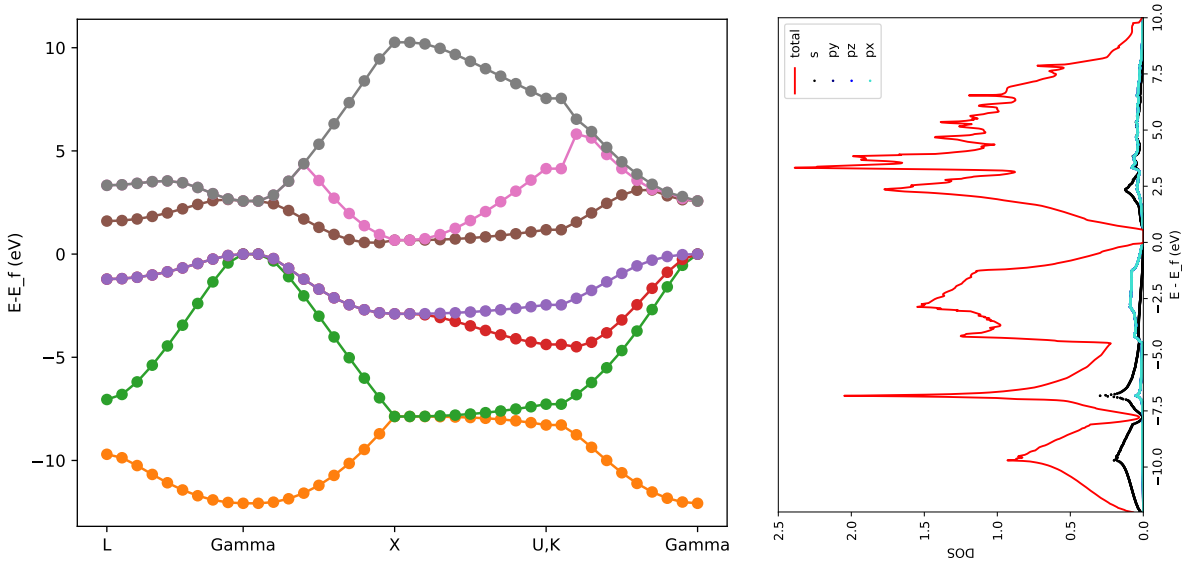

Figure 2: Bandstructure and DOS of Si

1.2 Bandstructure and DOS

Next, we calculate the electronic band structure and plot the energy per band in Fig. 2 on the left-hand side. On the right-hand side, we plot the DOS. Be aware that for the DOS calculation, you should not use the same input as from the band structure calculation! Specifically, for the DOS calculation the KPOINTS file should not be in line mode! Use the standard INCAR and then make modifications according to the instructions. For both plots, shift the energy scale so that the fermi energy is at 0. The band gap, which is indirect, is underestimated (we find a gap of around 0.7 eV which is lower than the literature value of around 1.1 eV, see for example `https://www.nature.com/articles/s41598-019-47670-y` and just standard text books for example on photovoltaic).

Otherwise, these plots look very similar to this reference: `https://lampz.tugraz.at/~hadley/memm/materials/silicon/silicon.php?print`

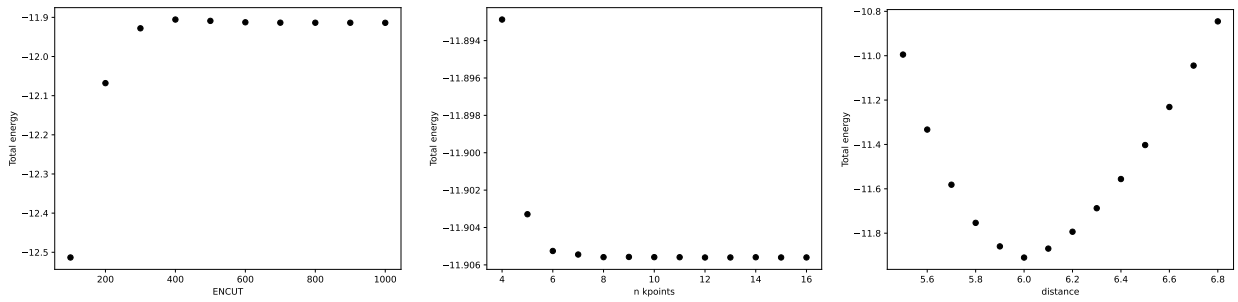

Figure 3: Convergence testing and lattice optimization for MgO

2 Phonon properties and electric susceptibility of MgO

The aim of this exercise is to 1. find the frequency of the phonons and 2. calculate the electric susceptibility for magnesium oxide MgO.

2.1 Convergence and geometric optimization

As before, we find the ENCUT, number of KPOINTS and the lattice constant by doing convergence testing and optimization. My takeaway is: ENCUT=600, n KPOINTS>8, lattice constant 6 Å.

2.2 Lattice dynamics

We find 3 acoustic branches with eigenvalue 0, and 3 optical ones with eigenvalue $\Omega \approx 11.8\,\mathrm{Hz}$. We find this by calculating the force matrix and thus the dynamical matrix as instructed. It really helps writing that matrix out by hand to see how it is constructed. Then I just use the python linalg librarys eig functions to calculate eigenvalues and -vectors, `https://numpy.org/doc/stable/reference/generated/numpy.linalg.eig.html`.

2.3 Dielectric properties

We have a look at two different parts of the susceptibility.
1. Electronic part of susceptibility
First the electronic part of the susceptibility as read out from the dielectric function. A plot of the real and imaginary part can be seen in Fig. 4. Remember that the susceptibility χ is linked to the dielectric constant ϵ with $\chi = \epsilon - 1$.
2. Lattice contribution to the electric susceptibility
The lattice dynamics in terms of phonon amplitude Q in response to an electric field E is described by a harmonic oscillator with

$$\ddot{Q}(t) + \kappa\dot{Q}(t) + \Omega^2 Q(t) = \sum_i Z_i E_i(t) \tag{1}$$

where ω is the frequency variable, κ the linewidth (damping) and Ω is the resonance frequency. On the right hand side, the sum runs over Cartesian coordinates $i = \{x, y, z\}$. E is the electric field and Z the components of the mode effective charge.

First, we want to find an analytical expression for the phonon amplitude in terms of electric field. We

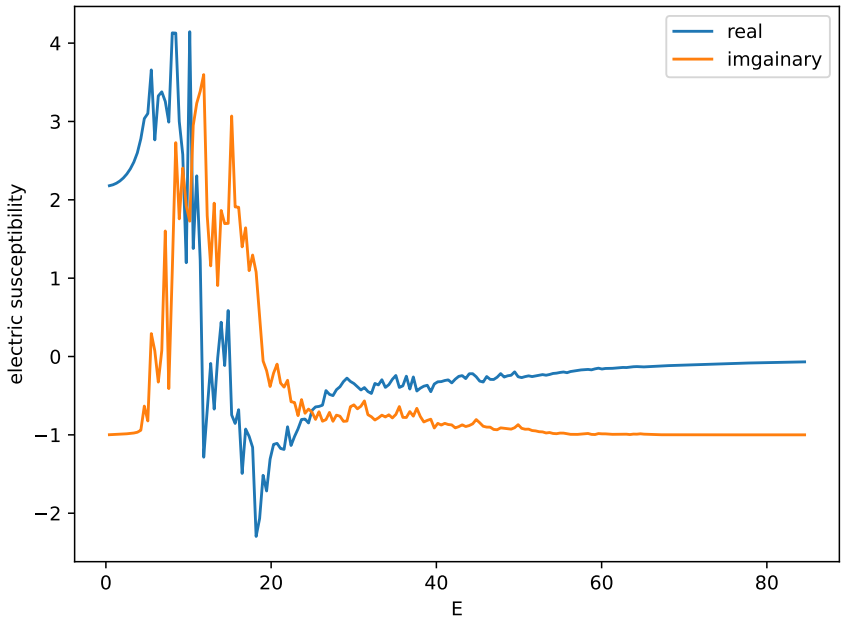

Figure 4: Electronic susceptibility

plug the ansatz

$$Q(t) = Q_\omega \exp(-\imath\omega t) \tag{2}$$

$$\dot{Q}(t) = -\imath\omega Q_\omega \exp(-\imath\omega t) = -\imath\omega Q(t) \tag{3}$$

$$\ddot{Q}(t) = (-\imath\omega)^2 Q_\omega \exp(-\imath\omega t) = -\omega^2 Q(t) \tag{4}$$

into Eq. (1) and find

$$-\omega^2 Q(t) - \kappa\imath\omega Q(t) + \Omega^2 Q(t) = \sum_i Z_i E_\omega \exp(-\imath\omega t) \tag{5}$$

$$\Leftrightarrow Q_\omega \left[-\omega^2 - \kappa\imath\omega + \Omega^2\right] = \sum_i Z_i E_{i,\omega} \tag{6}$$

$$\Leftrightarrow Q_\omega = \frac{\sum_i Z_i E_{i,\omega}}{-\omega^2 - \kappa\imath\omega + \Omega^2} \tag{7}$$

which leads to

$$\vec{p}_{ph} = \vec{Z}Q = Z_j \frac{Z_i E_{i,\omega}}{-\omega^2 - \kappa\imath\omega + \Omega^2} \tag{8}$$

when plugging into the definition of the phonon amplitude. Here, we use i and j to denote the cartesian components. χ is a 3×3 matrix, which is created by running both i and j over $\{x, y, z\}$.

Before, we've learned that the susceptibility is a tensor that connects the electric polarization $\vec{P}$ to ane applied electric field $\vec{E}$,

$$\vec{P} = \epsilon_0 \chi(\omega) \vec{E}(\omega) \tag{9}$$

with the vacuum permittivity ϵ_0 (constant), and $\chi(\omega)$ is the lattice part of the susceptibility.

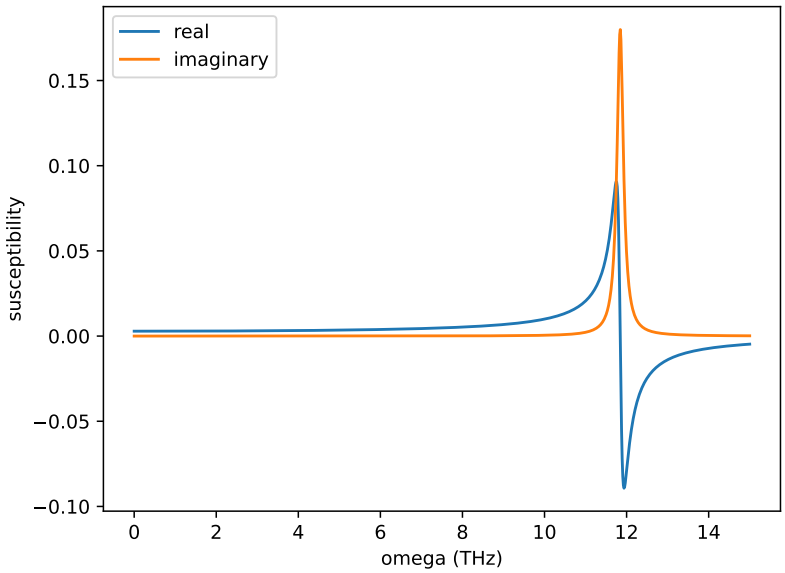

Figure 5: Lattice part of the susceptibility

By definition, the volume V_c has to be included for the polarization, $p_k = \int_{V_c} P_k dV_c \approx P_k V_c$. If we compare Eq. (8) to Eq. (9), we see that

$$\epsilon_0 V_c \chi(\omega) = \frac{Z_i Z_j}{-\omega^2 - \kappa i\omega + \Omega^2} \tag{10}$$

where i and j denote Cartesian components and run over $\{x, y, z\}$ and V is the volume. We have calculated $\Omega \approx 11.8\,\mathrm{THz}$ before.

Now, the mode effective charge (vector with 3 cartesian components) can be found with the Born effective charge tensor Z^* and the phonon eigenvectors q, with

$$Z_i = \sum_n Z^*_{n,ij} \frac{q_{n,j}}{\sqrt{M_n}}. \tag{11}$$

Here, $i \in \{x, y, z\}$ denotes the cartesian components of Z, n counts over all atoms involved (two in the case of MnO), and $j \in \{x, y, z\}$ also denotes cartesian components. M_n is the mass. The Born effective charge tensor can be calculated using VASP, using NCORE=1 and LEPSILON = .TRUE. (see exercise sheet). The matrix is found in the OUTCAR under BORN EFFECTIVE CHARGES for each atom. Once this is extracted, we multiply the according Z^* matrix with the phonon eigenvectors of the according atom that we have calculated in the previous task, (and divide by the mass), and thus obtain Z. Then, with Eq. (10), we can plot the real and imaginary part of the susceptibility, assuming $\kappa = 0.1\Omega/2\pi$. The cell volume V_c can be found in the OUTCAR file btw!

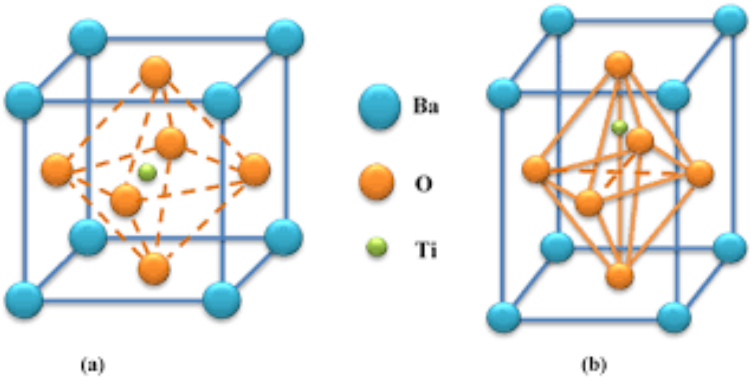

Figure 6: Two phases of $BaTiO_3$. Left: cubic (high temperature), no polarization. Right: tetragonal (low temperature), with finite polarization, thus ferroelectric. From 10.1007/s10832-021-00266-3

3 Ferroelectric (FE) polarization in BaTiO3

3.1 FE polarization in the tetragonal phase

We will calculate the polarization of the perovskite-oxide $BaTiO_3$. There is a structural FE phase transition where a polarization is allowed due to lower lattice symmetry: the Ti is shifting away from the center of the cell. Shifting up or down $\rightarrow$ positive or negative polarization.

First we calculate the polarization for the tetragonal phase with

$$P_i = \frac{1}{V_c} \sum_n Z^*_{n,ij} u_{n,j} \tag{12}$$

where V_c is the volume, $n = 1..5$ denotes the atom, i, j Cartesian components, and u the displacement. The latter we find by taking the difference in configuration between the tetragonal and cubic structure. With VASP we can calculate the Born effective charge tensor Z^*. Then, we matrix-multiply it with the displacements (taken from the 'Direct' structures). But be careful! When computing the polarization, we have to transform the displacements back to real positions, i.e. multiplying with the transformation matrix (the first matrix in the POSCAR file, which gives the tetragonal basis).

3.2 Tuning the polarization

There are two energy-degenerate solutions for the tetragonal phase; one with negative, and one with positive polarization, corresponding to the shift of the Ti atom in one or the other direction. We now aim to the energy landscape when transitioning from one to the other phase. For that, we calculate the difference between cubic and tetragonal structure (difference in the position matrix, the latter one in the POSCAR file), and move from - tetragonal to 0 (corresponding to cubic) to + tetragonal in discrete steps. Also, to make sure that the tetragonal configuration is really in an energy minimum, we transform/displace even further to either side. The resulting picture shows a double-well potential landscape, see Fig. 7. Here, the energy scale is shifted by the off-set energy at 0 polarization. We fit the data with

$$E = aP^2 + bP^4, \tag{13}$$

see Fig. 7 on the left, but notice that this is not enough. However, using

$$E = aP^2 + bP^4 + cP^6 \tag{14}$$

see Fig. 7 on the right, we get a nice fit!

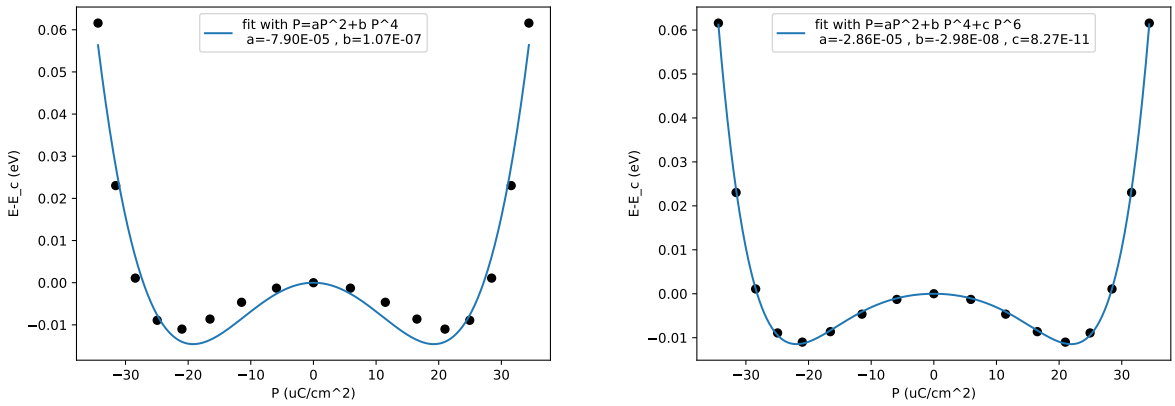

Figure 7: Energy landscape when deforming the tetragonal structure. Left: fitting with a function up to fourth order, right: fitting with a polynomial up to 6th order.

Note that $a < 0, b > 0$ only when using the polynomial to fourth order.

The height of the barrier is around $\Delta V = 0.01\,\mathrm{eV}$. With that, the electronic field strength necessary to overcome this barrier is around $E_c = -\Delta V / p_x$ with $p_x = P_x V_c$. Here, V_c refers to the cell volume, and E the electric field strength. P_x is the polarization we have calculated to be $21\,\mu\mathrm{C\,cm^{-2}}$. The cell volume is $63.9489\,\mathrm{\AA}^3$ (can be found in OUTCAR as 'volume of cell'). With that, we find that an electric field of $V_{edc} = 1.27 \times 10^8\,\mathrm{V\,m^{-1}}$. This is comparable to this article `https://www.sciencedirect.com/science/article/pii/S0927025613005375` given that they forgot a factor 10 in their result.

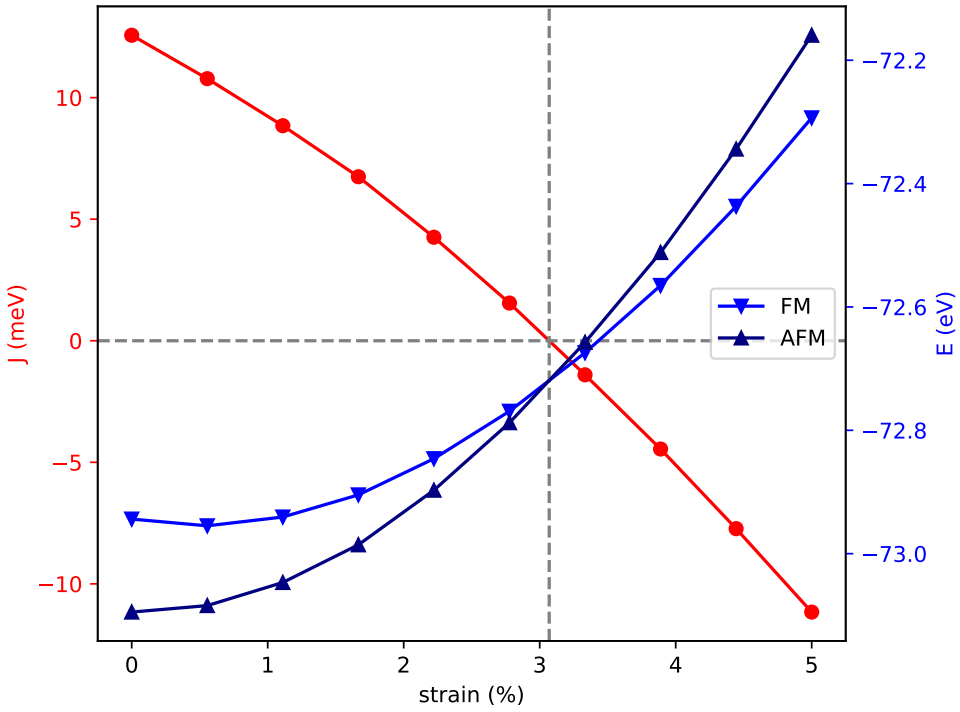

Figure 8: Energies for FM vs AFM configuration and resulting coupling constant J. Strain tunes the system from FM to AFM.

4 Magnetization of SrMnO

In this exercise, we calculate the magnetic properties of the AFM SrMnO. For the unstrained case, we find $J = 12.5\,\text{meV}$, which is comparable to `https://doi.org/10.1103/PhysRevMaterials.2.104409` (they find 13meV for first NN interaction). We however ignore higher neighbors and anisotropies; we estimate the exchange with

$$J = \frac{E_{FM} - E_{AFM}}{12} \tag{15}$$

where the 12 comes from the amount of neighbors. Strain can change the magnetic ground state. We simulate it by scaling the lattice constant (first scalar in the POSCAR file). From Fig. 8 we see that at around 3.1% strain, the ground state then changes from AFM to FM; the coupling J changes sign.

5 Handy things I have learnt when setting up the scripts

5.1 VASP

5.2 bash, grep, sed etc

Easy but handy things. Simple things are controlled with bash etc, more difficult operations I do with python scripts.

Listing 1: Job script for Umbrella (TU/e cluster)

```
!/bin/bash
#SBATCH --job-name=test_VASP
#SBATCH --output=log
#SBATCH --partition=tue.default.q
#SBATCH --time=01:00:00
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=2G

module load VASP/6.4.2-foss-2023a-VASPsol-VTST

mpirun -np 1 vasp_std
```

Listing 2: Easy bash loop

```
for i in {1..5};
do
        #creates directory
        mkdir testEnConv_$i
        #copy input files into directory
        cp jobscript POSCAR INCAR KPOINTS POTCAR testEnConv_$i
        #walk into directory
        cd testEnConv_$i
        #write new energy cut into INCAR file
        #this appends the 'string' to the end of the >> file
        # (( )) is for math operations, only easy math and only integers!!
        echo 'ENCUT='$((300+i*50)) >> INCAR
        #send to cluster
        sbatch jobscript
        # don't forget to come back!
        cd ../
done
```

Listing 3: Read out and modify with grep

```
for i in {0..10..1}
do
        #with grep you can extract the string after a certain pattern
        # copy it into a >> file
        grep "band     $i" PROCAR >> energies_band$i
        #or, when you are looking for the total free energy:
        # grep "free  energy   TOTEN*" OUTCAR >> ../energiesPositive_direct
        # this variateion copies the three lines follwing the pattern
```

```
        # then it cuts out all the lines with the given pattern
        # like that I get the raw data
        # grep -A 3 "TOTAL-FORCE" OUTCAR | grep -v "TOTAL-FORCE" >> ../Forces
        #alternatively, use sponge to take out lines with certain pattern like
done
```

Listing 4: Read out and modify with grep II

```
for i in {0..8..1};
do
        echo $i
        cd Positive_$i
        #extract BEC matrix from OUTCAR and copy into new file
        grep -A 21 "BORN EFFECTIVE CHARGES (including local field effects)
            (in |e|, cummulative output)" OUTCAR
            | grep -v "BORN EFFECTIVE CHARGES
            (including local field effects) (in |e|, cummulative output)"
            >> ../PositiveBEC_direct_$i.txt
        # now I take out all lines that contain the word 'ion'
        grep -v ion ../PositiveBEC_direct_$i.txt
            | sponge ../PositiveBEC_direct_$i.txt
        cd ../
done
```

Listing 5: replace things with sed

```
for i in {4..6..1};
do
        for j in {1..9..1};
        do
                mkdir testDistance_$i$j
                cp jobscript POSCAR INCAR KPOINTS POTCAR testDistance_$i$j
                cd testDistance_$i$j
                #write new distance into POSCAR file
                # search and replace with
                # sed 's/what to search/what to replace'
                # One big problem with bash is it can only do integers
                #(also true when doing calculations)
                # this is my easy but annoying work-around:
                sed -i "s/!replace/$i.$j/" POSCAR
                #send to cluster
                sbatch jobscript
                cd ../
        done
done
```